

Hafta 2 — Derste Yapılacak Uygulamalar

Ortam: Sınıfta öğrenci bilgisayarı **yok**. Tüm uygulamalar **hoca ekranında canlı** yapılır; öğrenci tahmin eder, hoca çalıştırıp doğrular. Kodlama öğrenciye **ev görevi** olarak devredilir. **Toplam süre:** 38 dakika — blok sürelerinin **içindedir**, ek süre değildir (bkz. [H02-anlatim.md](#) §2) **Gerekli:** Projeksiyon, terminal, Hafta 1'in [KampusKitap](#) projesi. İnternetsiz yedek planlar aşağıda.

Hazırlık — Derse girmeden önce (5 dk, ders dışı)

- ☐ [KampusKitap](#) projesi `dotnet run` ile açılıp kapatıldı (çalıştığı doğrulandı)
- ☐ Projenin **Hafta 1 hâlinin bir kopyası** alındı (canlı değişiklikler geri alınacak)
- ☐ Terminal yazı tipi ≥ 18 punto, tema açık renk
- ☐ Tarayıcıda yalnızca bir sekme; adres çubuğu rahat görünüyor
- ☐ [H02-etkilesimli.html](#) açıldı, `N` ile notlar denendi ve **kapatıldı**

Uygulama A — Controller Ekleyip Route'u Keşfetmek

Süre: 14 dakika · **Blok:** B **Amaç:** Hafta 1'in cevapsız bıraktığı soruyu (*sunucu hangi kodu çalıştıracakını nereden bildi?*) canlı olarak cevaplamak; route kalıbının **tahmin edilebilir** olduğunu göstermek.

Adım 1 — Boş bir controller ekle

`Controllers/KitapController.cs` :

```
using Microsoft.AspNetCore.Mvc;

namespace KampusKitap.Controllers;

public class KitapController : Controller
{
    public IActionResult Index()
    {
        return Content("Kitap listesi");
    }
}
```

```
dotnet run
```

Sınıfa sor (tahmin ettir, sonra dene): "Bu kodu yazdım. Tarayıcıda hangi adrese gidersem bu metin çıkar?"

Beklenen cevaplar: `/Kitap`, `/Kitap/Index`. **İkisi de doğru** — göster.

⚠ **Sık hata:** `namespace` satırını projenin adıyla uyumsuz yazmak. Derlenir ama karışıklık yaratır.

Adım 2 — Sözleşmeyi kasten boz

Sınıf adını `KitapController` → `Kitaplar` yap (soneki sil).

Önce tahmin ettir: "Şimdi `/Kitap` ne döndürür? Hata mı, 404 mü, yine çalışır mı?"

Beklenen çıktı: **404.** Derleme hatası yok, uyarı yok.

Söylenecik cümle: "Çatı bu sınıfı görüyor ama controller saymıyor. Çünkü sözleşme şu: adı `Controller` ile bitmeli. Sözleşmeyi bozunca hata almıyoruz — sadece sayfa yok."

Adı geri al.

Adım 3 — İkinci action ve parametre

```
public IActionResult Detay(int id)
{
    return Content($"Kitap #{id} detayı");
}
```

Sınıfa sor: "`/Kitap/Detay/42` ne döndürür?" → `Kitap #42 detayı`

Devam sorusu: "`/Kitap/Detay` (id vermeden) ne döndürür? Hata mı?"

Beklenen çıktı: `Kitap #0 detayı` — hata değil, sıfır.

Vurgulanacak nokta: "Route'ta `{id?}` opsiyonel ama parametre `int`. Değer gelmeyince C#'ın varsayılanı olan 0 kullanılıyor. Sessiz tuzak — ders notu §3.2."

Adım 4 — Kısıt ekle

`Program.cs` 'te route kalıbını `{id:int?}` yap.

Önce tahmin ettir: `"/Kitap/Detay/abc` şimdi ne döndürür?"

Beklenen çıktı: 404 — route eşleşmedi.

Bağla: "Kısıt koymak, hatayı 'sessiz 0' yerine 'açık 404'e' çeviriyor. Hangisi daha iyi? Neden?"

Kalıbı geri al.

Yedek plan (proje çalışmıyorsa): Etkileşimli sayfadaki **route çözümleyici** bölümü (*Route kalıbını çöz*) aynı adresleri tıklamayla çözdürür; tahmin ettirme aynen yapılır.

Uygulama B — Bir Servisi DI ile Enjekte Etmek

Süre: 12 dakika · **Blok:** C **Amaç:** `new` ile nesne üretmenin alternatifini göstermek; yaşam döngüsü farkını **sayaçla gözle görünür** kılmak.

Adım 1 — Servisi yaz

Services/ZiyaretSayaci.cs :

```
namespace KampusKitap.Services;

public interface IZiyaretSayaci
{
    int Arttir();
}

public class ZiyaretSayaci : IZiyaretSayaci
{
    private int _sayi = 0;
    public int Arttir() => ++_sayi;
}
```

Adım 2 — Kaydet ve enjekte et

Program.cs — `builder.Build()` 'dan önce:

```
builder.Services.AddSingleton<IZiyaretSayaci, ZiyaretSayaci>();
```

KitapController :

```
private readonly IZiyaretSayaci _sayac;

public KitapController(IZiyaretSayaci sayac)
{
    _sayac = sayac;
}

public IActionResult Index()
{
    return Content($"Kitap listesi - {_sayac.Arttir()}. ziyaret");
}
```

Sınıfa sor: "Ben hiçbir yerde `new ZiyaretSayaci()` yazmadım. Nesne nereden geldi?"

Sayfayı birkaç kez yenile: sayı **artarak** gider (1, 2, 3...).

Adım 3 — Yaşam döngüsünü değiştir, farkı göster

`AddSingleton` → `AddScoped` yap.

Önce tahmin ettir: "Şimdi sayfayı yenilediğimde ne göreceğim?"

Beklenen çıktı: her yenilemede **1** — çünkü her istek için yeni nesne üretiliyor.

Söylenecek cümle: "Aynı kod, tek kelime değişti, davranış tamamen farklı. Bu yüzden `DbContext` 'i Hafta 8'de `AddScoped` ile kaydedeceğiz: bir isteğin başından sonuna aynı bağlantı, istek bitince kapanıyor."

⚠ Sık hata (öğrenciler evde yaşayacak): Servisi `builder.Build()` 'dan **sonra** kaydetmek. Derlenir, çalışır, ama `Unable to resolve service` hatası alınır. **Bunu canlı göster** — satırı aşağı taşı, hatayı çıkar, geri al.

Yedek plan: Etkileşimli sayfadaki **yaşam döngüsü karşılaştırması** bölümü üç kaydı (Singleton / Scoped / Transient) yan yana simüle eder.

Uygulama C — Middleware Pipeline'ı Kasten Bozmak

Süre: 12 dakika · **Blok:** C **Amaç:** Haftanın ilkesini ("sırayı bozmak sessizce yanlış davranış üretir") **kanıtlamak**. Bu, dersin en akılda kalan kısmıdır — kesmeyin.

Adım 1 — Günlükleyen bir middleware ekle

`Program.cs` içinde, `app.UseHttpsRedirection();` satırının **hemen üstüne**:

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"→ GİRİŞ: {context.Request.Method} {context.Request.Path}");
    await next();
    Console.WriteLine($"← ÇIKIŞ: {context.Response.StatusCode}");
});
```

```
dotnet run
```

Tarayıcıda sayfayı yenile, **terminale bak**.

Beklenen çıktı:

```
→ GİRİŞ: GET /
← ÇIKIŞ: 200
→ GİRİŞ: GET /css/site.css
← ÇIKIŞ: 200
...
```

Sınıfa sor: "Tek sayfa açtım. Neden birden fazla satır yazdı?" → Her varlık (CSS, JS, ikon) **ayrı bir istektir**. Hafta 1'deki "bir sayfa tek istek değildir" tespitinin sunucu tarafındaki karşılığı.

Adım 2 — `await next()` 'i sil, short-circuit'i göster

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"→ GİRİŞ: {context.Request.Method} {context.Request.Path}");
    // await next();          ← SİLİNDİ
    Console.WriteLine($"← ÇIKIŞ: {context.Response.StatusCode}");
});
```

Önce tahmin ettir: "Bu değişiklikten sonra sayfayı yenilersem ne göreceğim?" Seçenekleri tahtaya yaz: (a) aynı sayfa, (b) hata sayfası, (c) bomboş sayfa, (d) çöker.

Çalıştır ve yenile.

Beklenen çıktı: Bomboş bir sayfa, durum kodu `200`. Hata yok, çökme yok.

Söylenecek cümle: "Uygulama çökmedi. Hata da vermedi. Sadece hiçbir şey yapmadı. Pipeline'da bir halka `next()` çağırmasa, arkasındaki her şey yok sayılır."

`await next();` satırını geri al.

Adım 3 — Sırayı boz: `UseAuthorization` önce gelsin

```
app.UseAuthorization();    // ← şimdi ÖNCE
app.UseRouting();          // ← şimdi SONRA
```

Önce tahmin ettir: "Uygulama çöker mi, sayfa açılır mı?"

Çalıştır, sayfayı yenile.

Beklenen çıktı: Sayfa normal açılır. Hiçbir hata yoktur.

Vurgulanacak nokta — haftanın ilkesi burada: "Şu anda yetkilendirme, hedefin ne olduğunu bilmeden çalışıyor. Bu projede korunan sayfa olmadığı için fark etmiyoruz. **Hafta 11**'de `[Authorize]` eklediğimizde bu sıra bozuk kalırsa korumamız çalışmayacak — ve **yine hata almayacağız**. Sessizce korumasız bir site yayınlayacağız."

Sırayı geri al.

⚠ **Zaman sıkışırsa:** Adım 3 kesilebilir ama **Adım 2 kesilmez** — short-circuit kavramı Hafta 5 (CSRF) ve Hafta 11 (Authorization) için ön koşuldur.

Yedek plan (proje çalışmıyorsa): Etkileşimli sayfadaki **pipeline'i bozmak** bölümü aynı deneyi tıklamayla yaptırır: sıra ve `next()` düğmeleriyle üç durumu da gösterir.

Ders Sonrası Öğrenci Görevi

`KampusKitap` projesinde:

1. **İki controller** ekleyin: `KitapController` ve `KategoriController`
2. Her birine **üçer action** yazın (`Content()` döndürebilirler)
3. Şu beş adresin hangi controller/action'a düşeceğini **çalıştırmadan** yazın, sonra doğrulayın: `/` · `/Kitap` · `/Kitap/Detay/5` · `/Kategori/Ara` · `/Kitap/Detay/abc`
4. Uygulama C Adım 1'deki **günlükleyen middleware'i** ekleyin
5. **Teslim:** tahmin listesi + gerçek sonuçlar + terminal çıktısının ekran görüntüsü

İsteğe bağlı (yapana artı): Uygulama B'deki `IZiyaretSayaci` 'yı kendi projenizde kurun ve `AddSingleton` ↔ `AddScoped` farkını ekran görüntüsüyle gösterin.

Hocaya Kontrol Listesi

Ders öncesi

- ☐ Projenin Hafta 1 hâli yedeklendi (canlı değişiklikler geri alınacak)
- ☐ `dotnet run` çalışıyor, port çakışması yok
- ☐ Terminal ve tarayıcı yan yana görünüyor (middleware çıktısı ile sayfa aynı anda)

Ders içi

- ☐ **Uygulama A Adım 2, 3, 4'te önce tahmin ettirildi** — sonra çalıştırıldı
- ☐ Uygulama B Adım 3'te sayaç farkı **canlı** gösterildi (1,2,3 → 1,1,1)
- ☐ **Uygulama C Adım 2 yapıldı** (kesilmemesi gereken tek adım)
- ☐ `builder.Build()` sonrası servis kaydı hatası canlı gösterildi
- ☐ Değiştirilen tüm satırlar **geri alındı**

Ders sonrası

- ☐ Ev görevi duyuruldu; Hafta 3'ün "düşünerek gelin" sorusu (`@RenderBody()`) söylendi



Yedek Planların Özeti

Sorun	Ne yapılır
Proje açılmıyor	Etkileşimli sayfadaki route çözümleyici + pipeline bozucu bölümleri kullanılır
Terminal çıktısı okunmuyor	Yazı tipini büyüt; ya da etkileşimli sayfadaki hazır terminal çıktısı gösterilir
Süre taşı	Uygulama B Adım 3 → Uygulama C Adım 3 → Uygulama C Adım 2 asla atlanmaz
Hiçbir şey çalışmıyor	Üç uygulamanın tamamı etkileşimli sayfadan tıklamayla yapılır